

uOttawa

FALL 2026

SEG3102 Software Design and Architecture

Domain-Driven Design: Structure design

Slide content used in this course is adapted from Prof. Stéphane Somé.

Last lecture: BDD

Explicit data, implicit behavior

```
Scenario Outline: Ordering depends on the available stock
  Given a signed-in customer with <in cart> <item> in the cart
  And <available> <item> available in stock
  When the customer places the order
  Then the order is <outcome>
  And <remaining> <item> remain available in stock

  Examples:
    | in cart | available | outcome    | remaining |      item      |
    | 2       | 10        | placed     | 8         | Wireless mouse |
    | 5       | 5         | placed     | 0         | Wireless mouse |
    | 6       | 5         | not placed | 5         | Wireless mouse |
```

Implicit data (data hidden in code), explicit behavior

```
Scenario: The available stock covers the ordered quantity
  Given a signed-in customer with items in the cart
  And the available stock covers the quantity of each item
  When the customer places the order
  Then the order is placed
  And the available stock of each item is reduced by the ordered quantity

Scenario: The available stock does not cover the ordered quantity
  Given a signed-in customer with items in the cart
  And the available stock does not cover the quantity of an item
  When the customer places the order
  Then the order is not placed
  And the available stock is unchanged
```

Last lecture: structure design

```
Command: placeOrder(customer, orderInfo)

Preconditions:
  - signed-in customer (non null)
  - the customer's cart has at least one cart line
  - for each cart line, the available stock covers the quantity

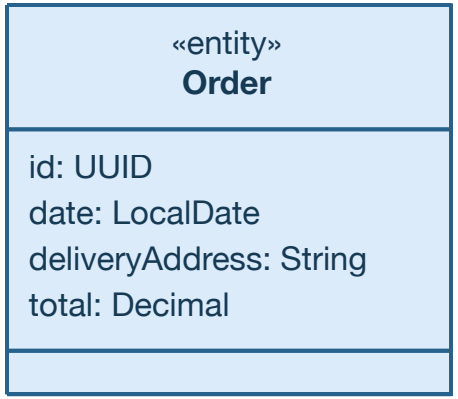
Post-conditions:
  - a new Order ord was created (creation of instance)
  - ord was initialized from orderInfo (modification of attributes)
  - ord was added to the customer's orders (creation of association)
  - for each cart line, a new Order Line ln was created from it
      (creation of instance, modification of attributes)
  - ln was added to ord (creation of association)
  - the available stock of each product was reduced by the ordered
      quantity (modification of attributes)
  - the cart lines were removed from the cart (destruction of instances)
```

- Tests through behavior-driven development
- **Structure modeling of the domain**
  - Every noun is a candidate concept: Customer, Cart, Cart Line, Order, Order Line, Product, Stock.

Last lecture: Entity

Entity

An **entity** is a domain object defined by its identity. Its attributes can change during its life; its identity never changes.




```
class Order(
    val id: UUID,
    val date: LocalDate,
    var deliveryAddress: String,
    var total: BigDecimal
) {
    override fun equals(other: Any?) =
        other is Order && other.id == id

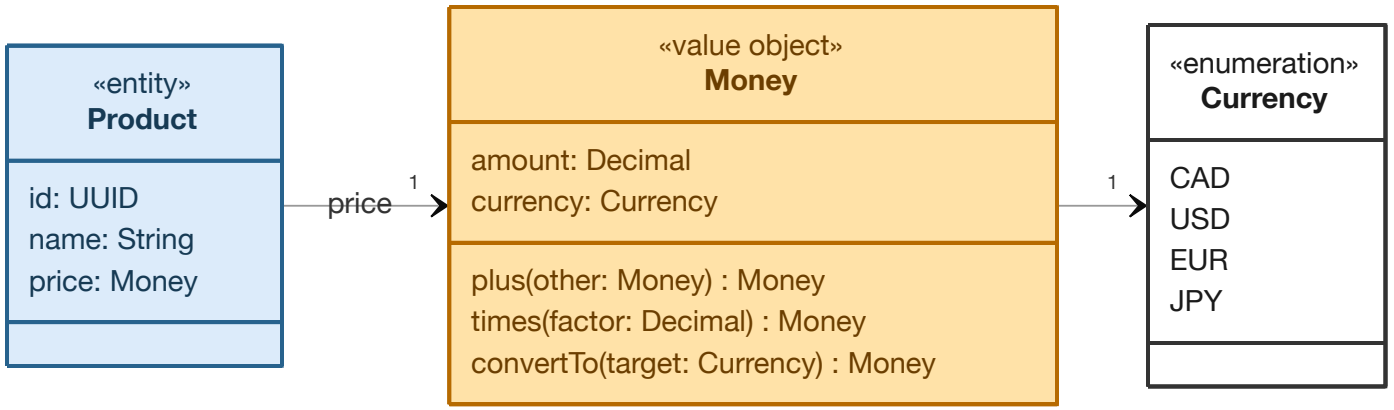
    override fun hashCode() = id.hashCode()
}
```

Equality is based on the `id`. Two orders with the same attributes are still two distinct orders, each with its own id.

Last lecture: Value object

Value object

A **value object** is a domain object defined only by its attributes. It has no identity and it never changes: to change a value, replace it with a new one.



Product, OrderLine, and Order all use `Money`. A new currency is one more value in `Currency`.

We can also use value object to implement business logic!

Software version

1.2.03

Semantic versioning

✓ attribute.str

1 . 2 . 03

major minor patch

✓

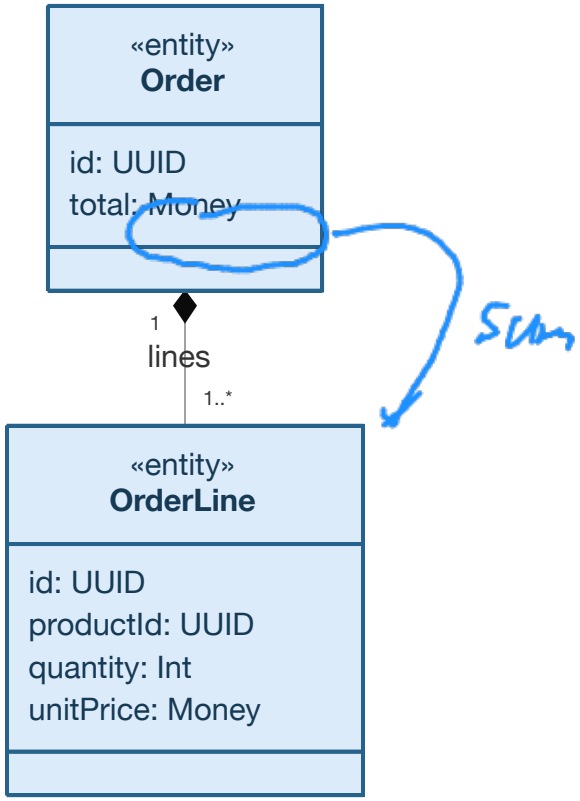


Outlines

- Aggregate
- Domain events
- Domain services
- Factories
- Repositories
- Putting it together

Aggregate

Customers want to change their order





Customers keep asking if they can change the quantity before their order ships?

Easy. An order line has a quantity; the `ChangeOrder` use case sets it.



```
// in the ChangeOrder use case
val line = order.lines.first { it.id == lineId }
line.quantity = 3
```

Something does not add up



Alright, let's test the `ChangeOrder` use case with a Gherkin scenario.

```
Scenario: The customer changes the quantity of an order line
  Given an order with 2 "Wireless mouse" at 20 dollars each
  And the order total is 40 dollars
  When the customer changes the quantity of "Wireless mouse" to 3
  Then the order has 3 "Wireless mouse"
  And the order total is 60 dollars
```

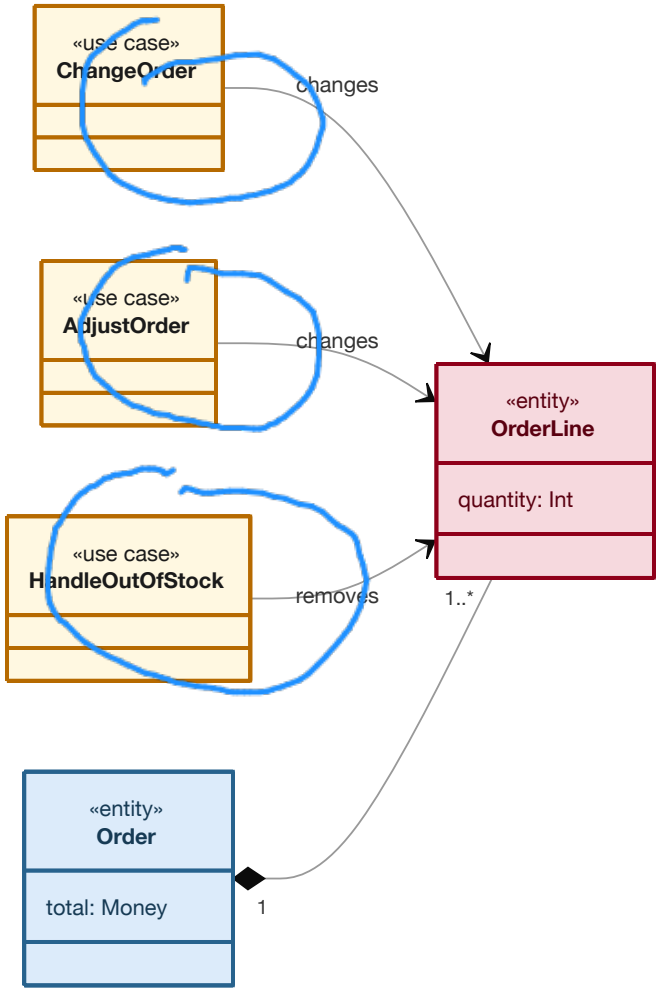
The last step fails: the total is still 40 dollars. Good thing the scenario caught it before an invoice went out.

```
// in the ChangeOrder use case
val line = order.lines.first { it.id == lineId }
line.quantity = 3
order.total = order.lines.map { it.subtotal() }.reduce(Money::plus)
```



Fixed, in `ChangeOrder`. The scenario passes.

A few months later



Three use cases now change order lines…

One more thing: an order always has at least one line.

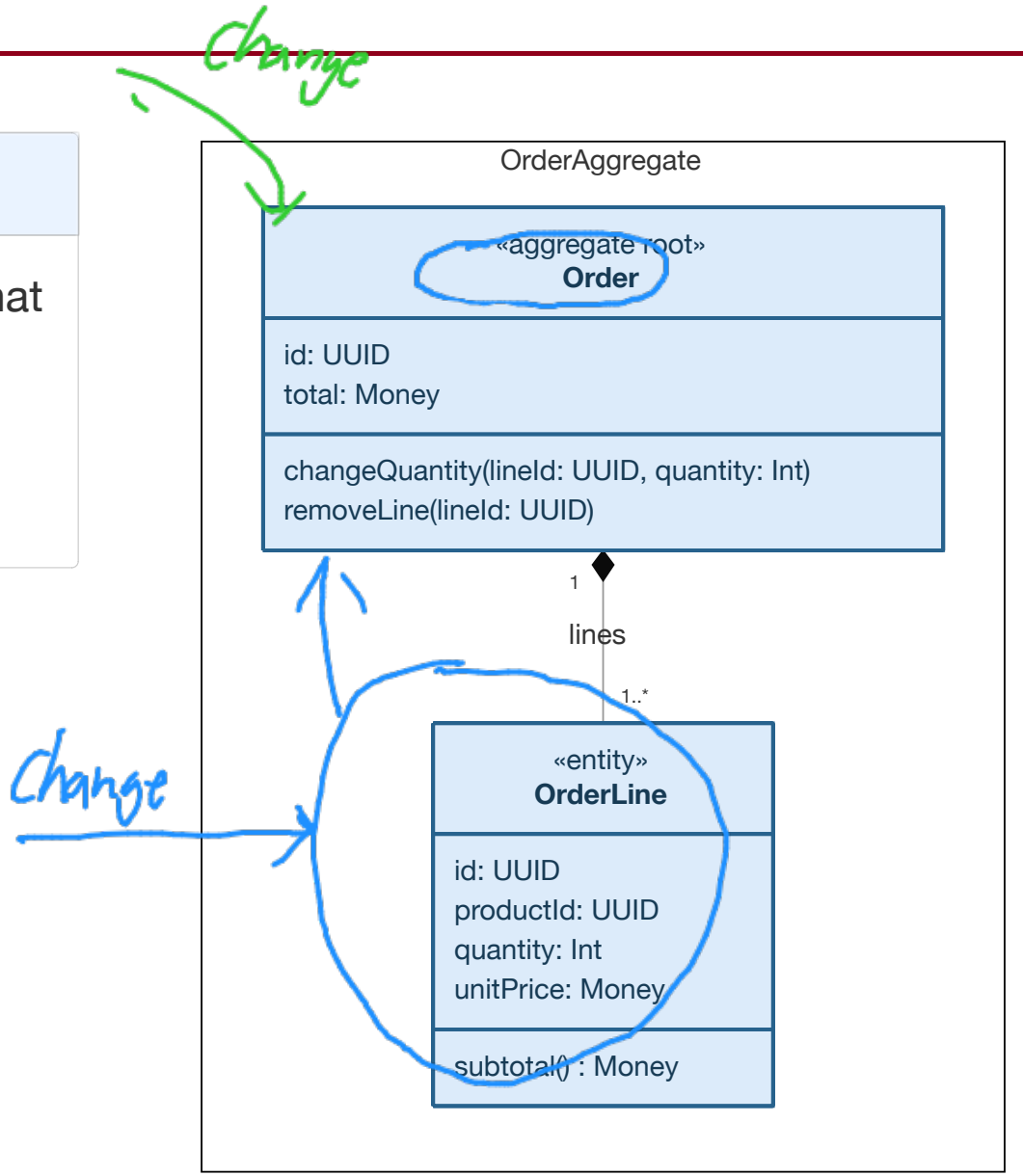
Three use cases must update the total and check that a line is left. Did anyone write one for `HandleOutOfStock`*?*

I'm pretty sure there will be more use cases like these coming… The rules belong to the order, but every code that touches an order line has to enforce them…

Aggregate

Aggregate

An **aggregate** is a cluster of entities and value objects that change together. One entity, the **aggregate root**, is the only entry point: outside code changes the cluster only through the root, and the root keeps the whole cluster consistent with its rules.



Change only through the root

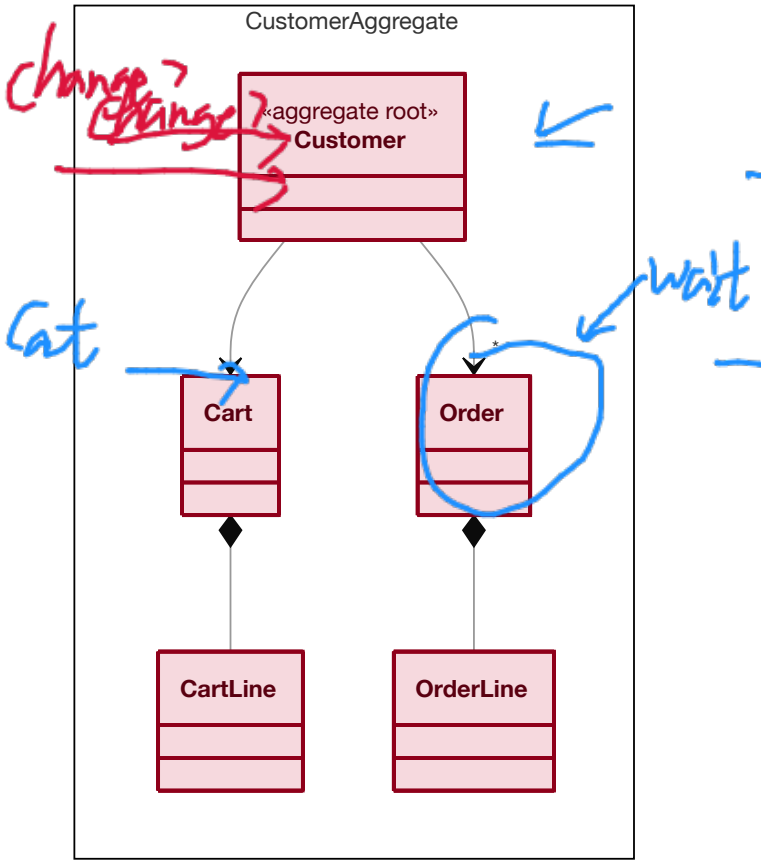
```
1  class Order(val id: UUID, /* ... */ lines: List<OrderLine>) {
2      private val _lines = lines.toMutableList()
3      val lines: List<OrderLine> get() = _lines    // read-only view
4
5      init {
6          require(_lines.isNotEmpty()) { "An order has at least one line" }
7      }
8      var total: Money = sum(); private set
9
10     fun changeQuantity(lineId: UUID, quantity: Int) {
11         _lines.first { it.id == lineId }.quantity = quantity  // internal setter
12         total = sum()
13     }
14
15     fun removeLine(lineId: UUID) {
16         require(_lines.size > 1) { "An order has at least one line" }
17         _lines.removeIf { it.id == lineId }
18         total = sum()
19     }
20
21     private fun sum() = _lines.map { it.subtotal() }.reduce(Money::plus)
22 }
```



The use cases can read the order lines, but every change goes through `Order`, which keeps the total right and the order non-empty.

Why not one big aggregate?

Customer owns the cart and the orders. Let's make Customer the root of everything: all the rules in one place.



Technically, this is not wrong. But now…

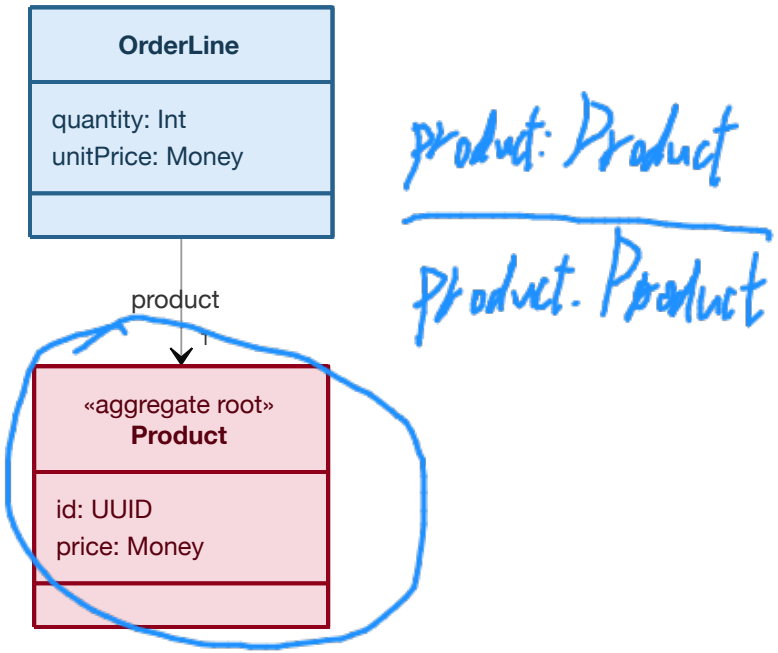
Loading a customer loads every order they placed in ten years.

If the operations are synchronous, while the warehouse updates one of their orders, adding a mouse to their cart has to wait.

Keep aggregates small: an aggregate holds only what must be consistent at the same moment.

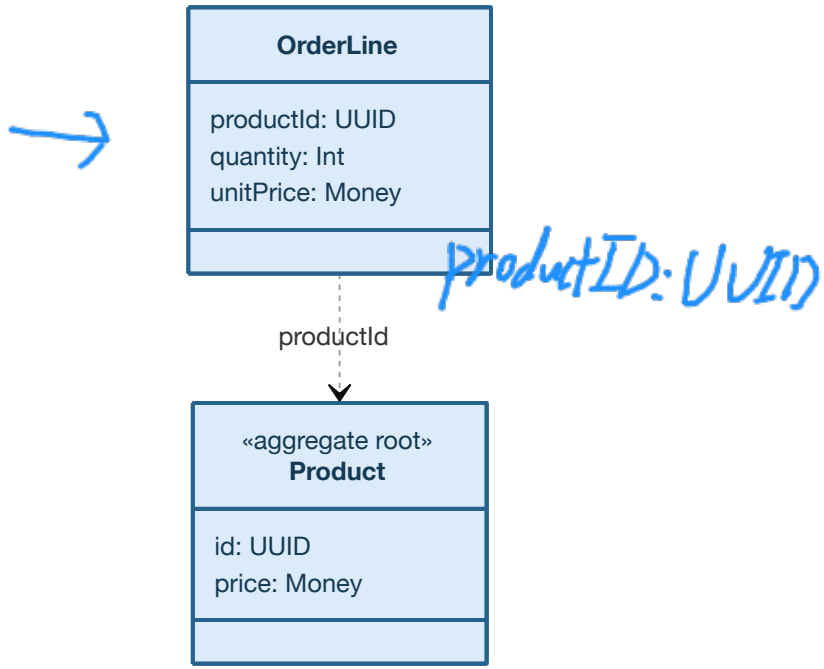
Reference other aggregates by id

Object reference



Code holding an order can change the product.
Loading an order also loads all product information.

Identity reference



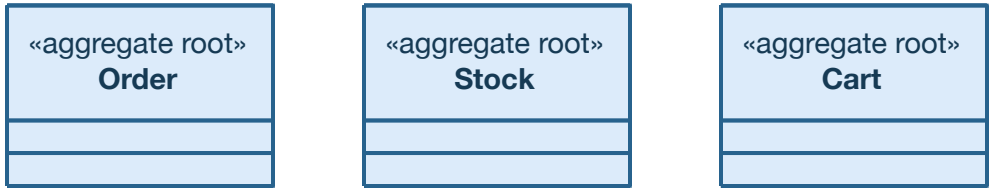
The order knows which product, and nothing more.
Each aggregate is loaded and changed on its own.

Prefer identity references between aggregates: hold the id of the other root, not the object reference.

One transaction, one aggregate

The contract of `placeOrder` changes three aggregates:

```
Post-conditions:
  - a new Order ord was created
  - ...
  - the available stock of each product was
      reduced by the ordered quantity
  - the cart lines were removed from the cart
```

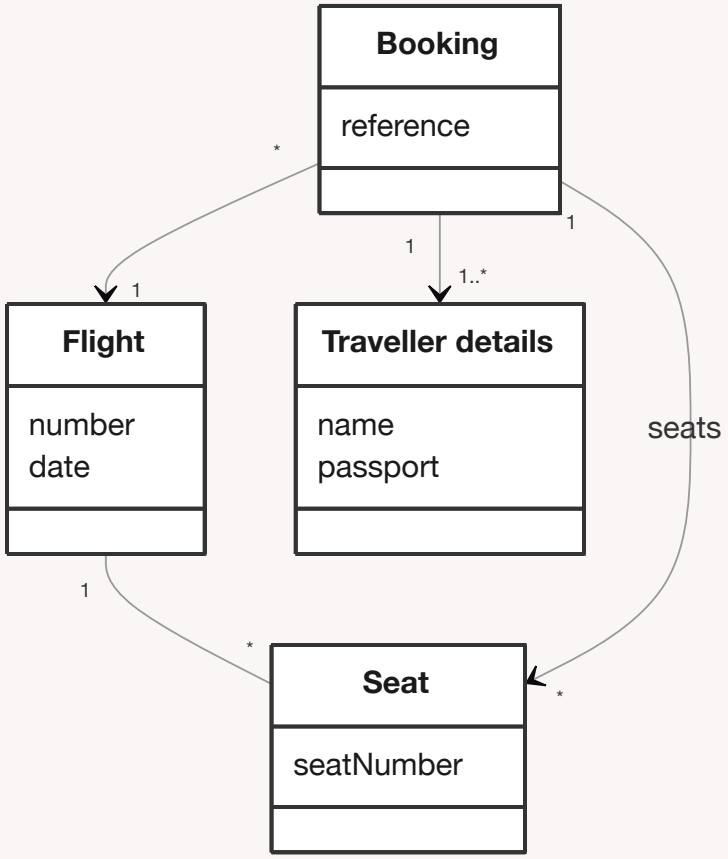


Since aggregate is the consistency boundary, one transaction (i.e. application command) changes one aggregate. So how do Stock and Cart learn that an order was placed?

best practice

The answer is: through event notifications.

Exercise: Where is the boundary?



A flight booking system has two rules:

1. Two travellers never hold the same seat on a flight.
2. A booking for several travellers gets seats for all of them at once, or for none.

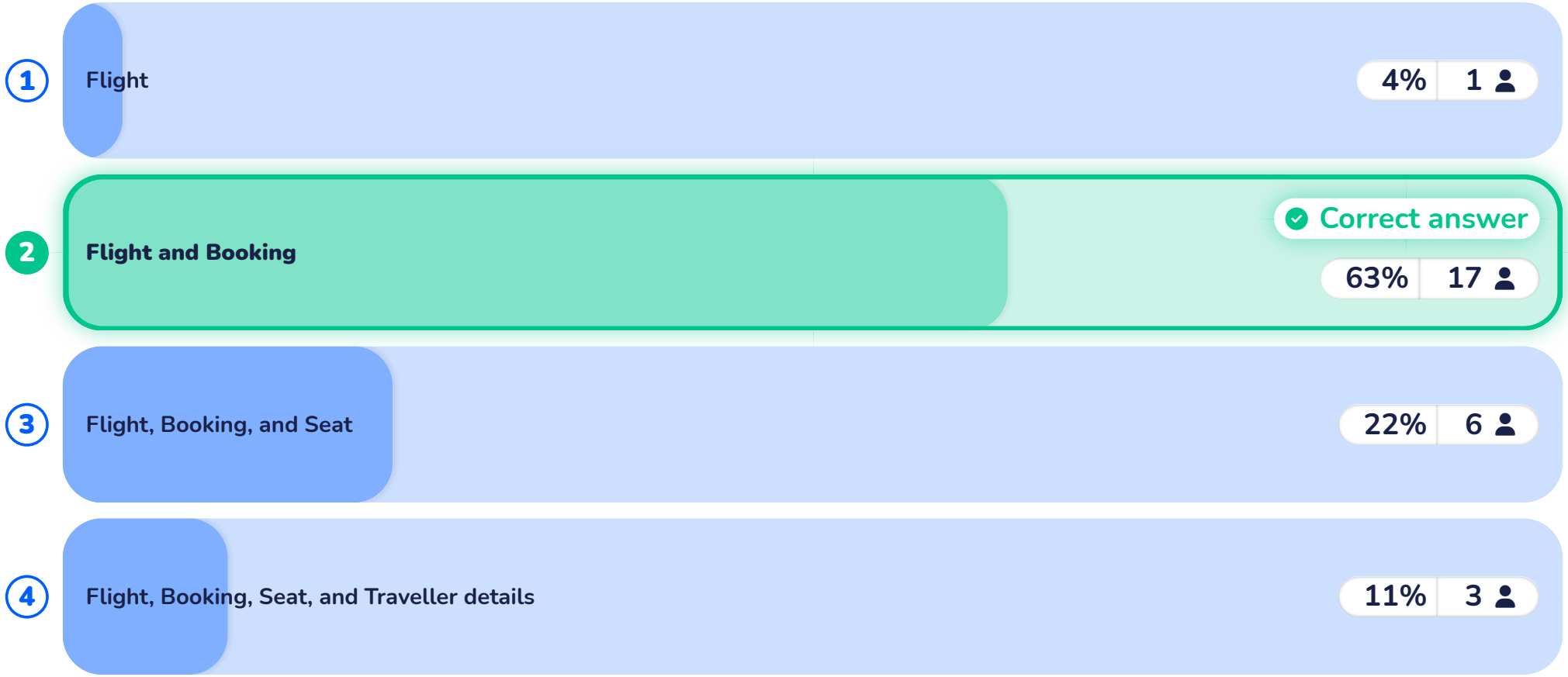
What are the aggregate roots in this model?

- A. Flight
- B. Flight and Booking
- C. Flight, Booking, and Seat
- D. Flight, Booking, Seat, and Traveller details

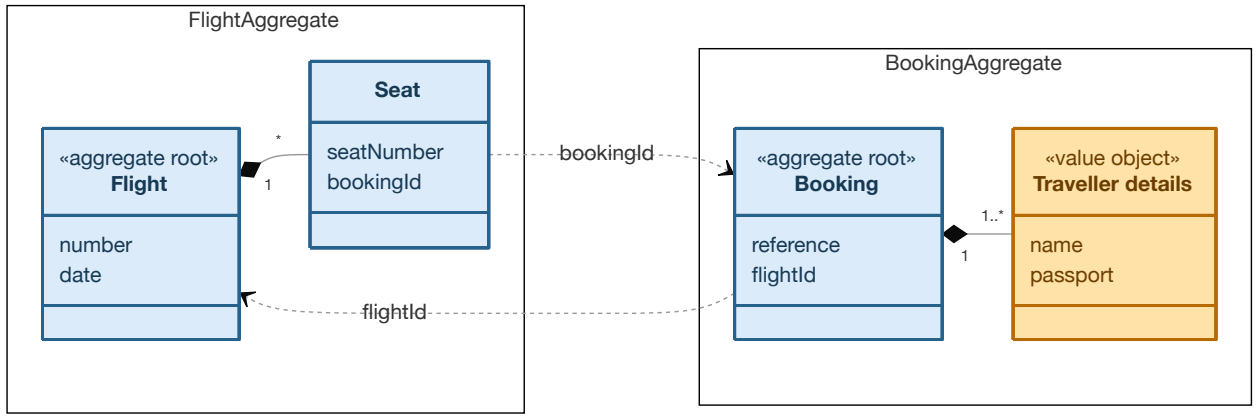
Exercise: Where is the boundary?

Go to wooclap.com and use the code **OCPCYJR**

A flight booking system has two rules: (1) two travellers...



Exercise: Answers



B. Rule 2 changes several seats at once, and rule 1 compares them with every other booking: only the Flight sees all its seats. Traveller details belong to one booking.

- A. Safe, but too big: every booking change locks the whole flight, and loading a flight loads hundreds of travellers.
- C. Seating a group touches several Seat aggregates in one transaction.
- D. Same problem as C, and traveller details mean nothing outside their booking.

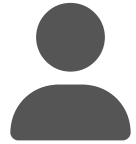
Domain event

The consequence propagates…



When an order is placed, we reduce the available stock.

And my cart should be emptied.



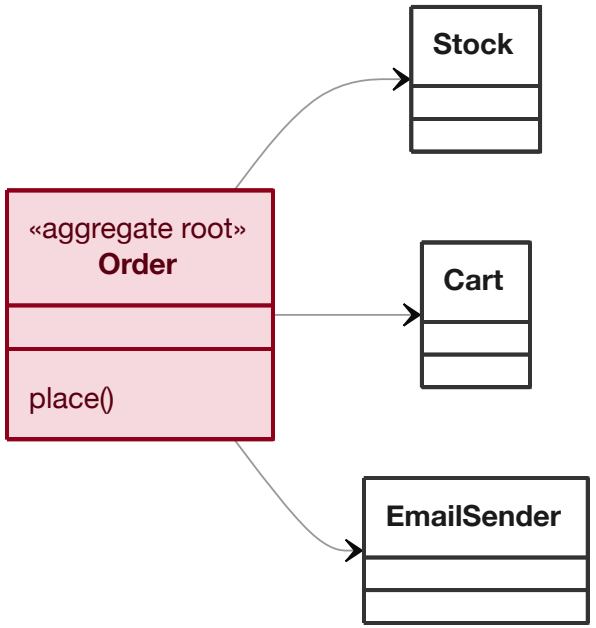


And we send the customer a confirmation email.

Order calls everyone

```
1  fun place() {
2      status = PLACED
3      stock.reduce(this)
4      cart.removeLines(this)
5      email.sendConfirmation(this)
6  }
```

Order





Next month, customers earn loyalty points on every order.

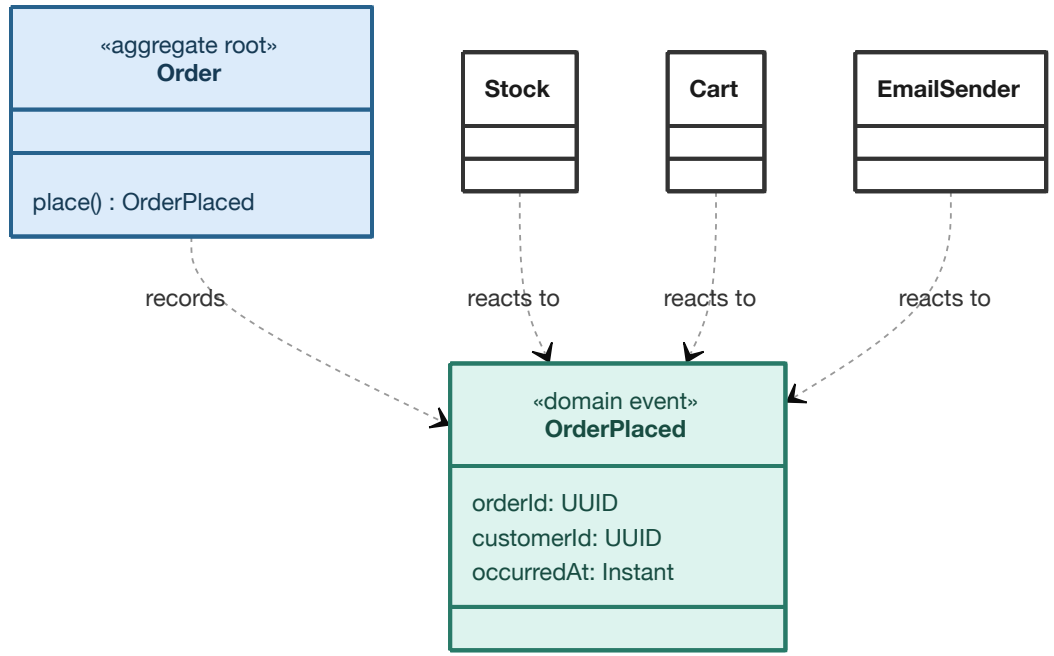
Order depends on three other classes, and may still increase…

Domain event

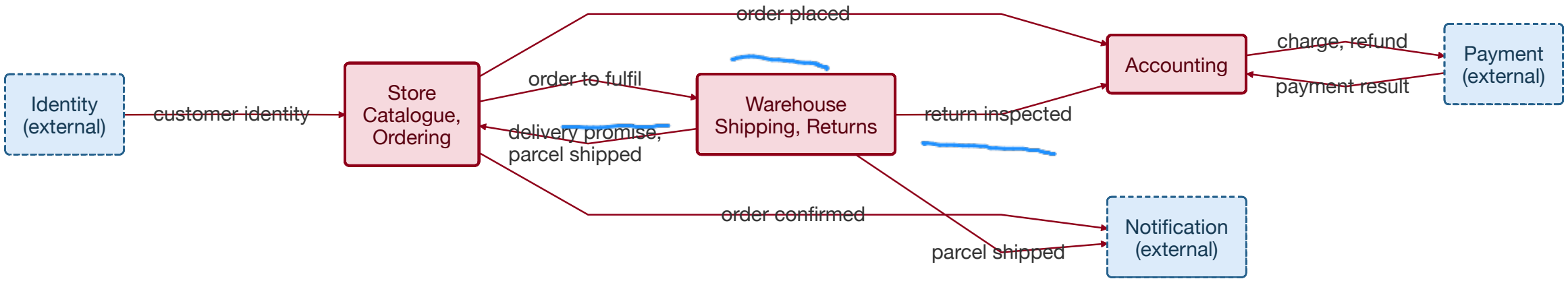
Domain event

A **domain event** records something that happened in the domain that the experts care about. It is named in the past tense, it never changes, and it carries the time and the identity of the objects involved.

```
1  data class OrderPlaced(
2      val orderId: UUID,
3      val customerId: UUID,
4      val occurredAt: Instant
5  )
6
7  fun place(): OrderPlaced {
8      status = PLACED
9      return OrderPlaced(id, customerId, Instant.now())
10 }
```

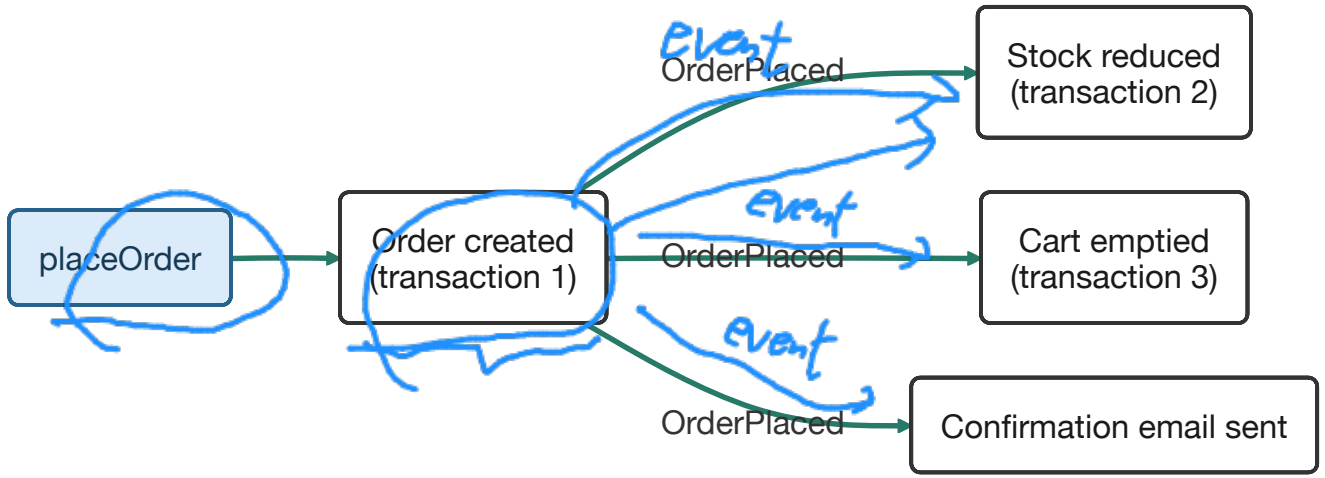


We have seen these before



"order placed", "parcel shipped", "return inspected": the context diagram was already full of domain events

Back to Stock and Cart



Each aggregate changes in its own transaction, and the email is sent, all in reaction to the event.

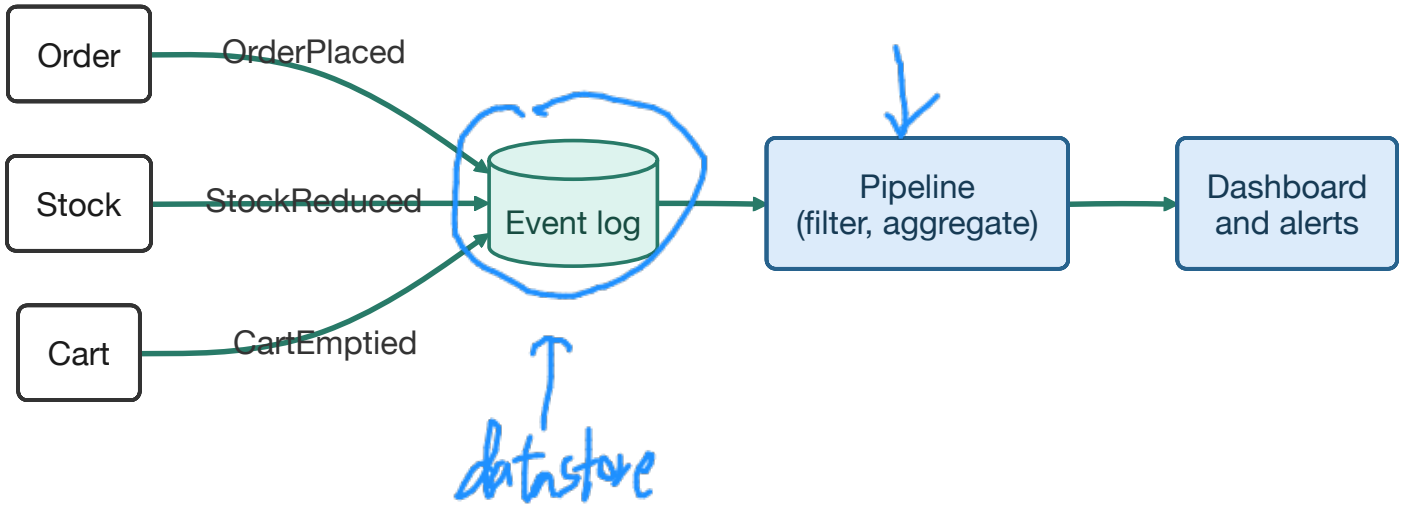
For a moment, the order exists and the stock is not yet reduced.

How to make sure the underlying data is consistent is a database/distributed systems problem.

Domain events for observability



How many orders were placed in the last hour? And how long does it take before the stock is reduced?

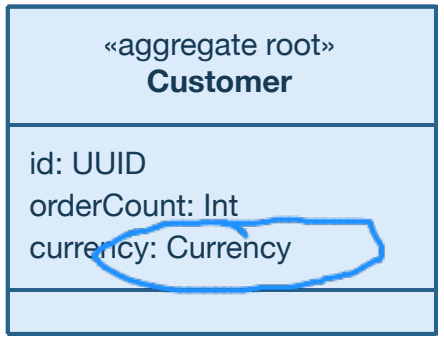


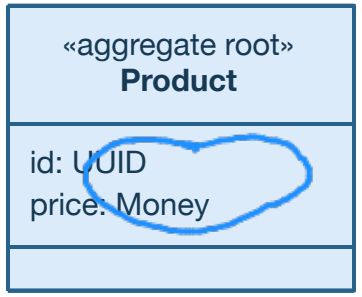
Domain service

The price for this customer



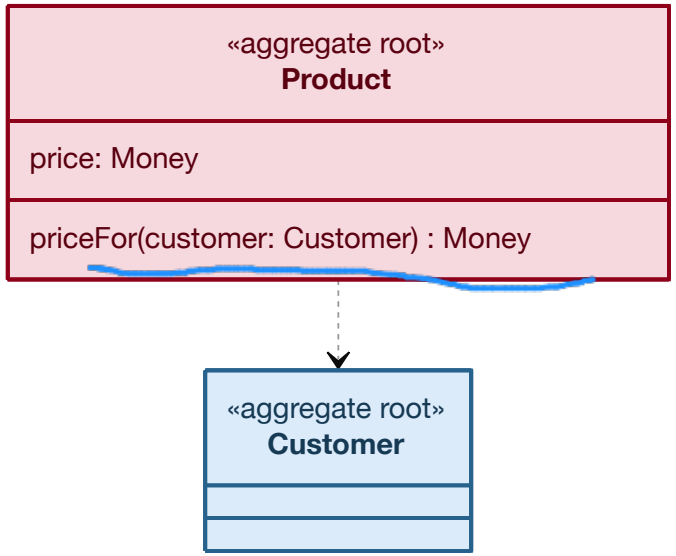
Customers with ten orders or more get 5% off. And each customer sees prices in their own currency.

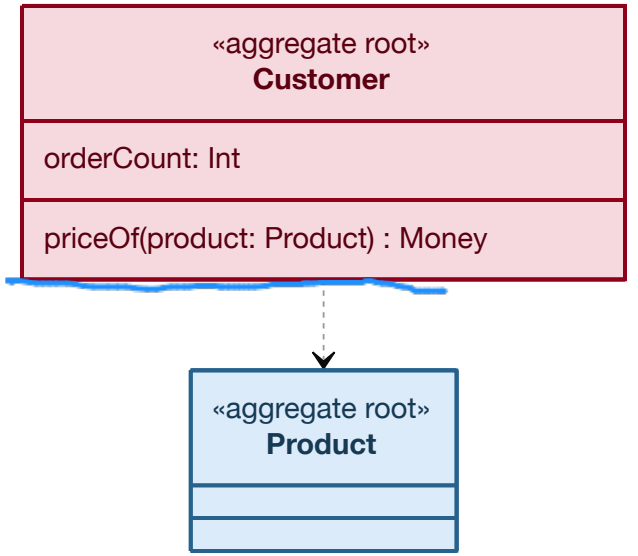




The rule needs the product's price and the customer's order count and currency. Which class should hold it?

Product or Customer?





The Product now knows about loyalty discounts.

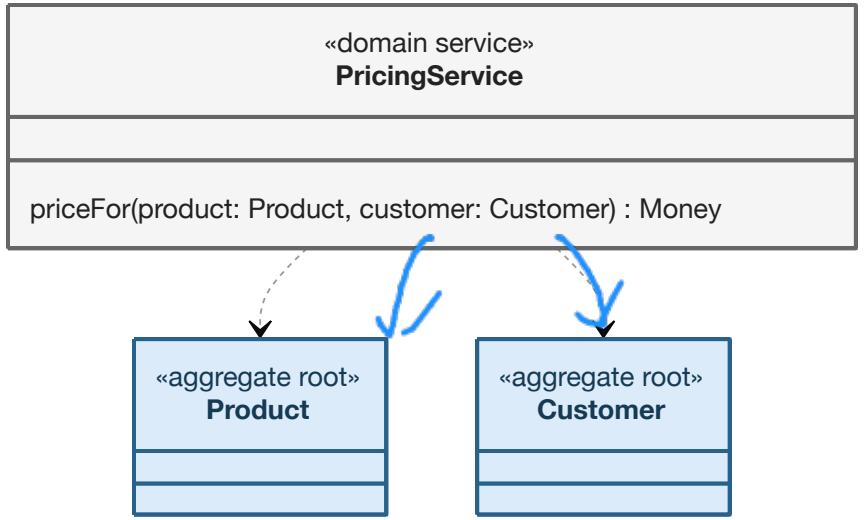
The customer now knows about product prices.

This rule is about a product and a customer together. It belongs to neither.

Domain service

Domain service

A **domain service** is a stateless operation that holds domain logic which does not belong to a single entity or value object. Its name comes from the ubiquitous language.




```kotlin
class PricingService {

    fun priceFor(product: Product, customer: Customer): Money {
        var price = product.price
        if (customer.orderCount >= 10) {
            price = price * BigDecimal("0.95")
        }
        return price.convertTo(customer.currency)
    }
}
```

no state.

The domain service is stateless: the same product and customer always give the same price.

Domain service or application command?

Application command

```
1  class PlaceOrderImpl(...) : PlaceOrder {
2      override fun placeOrder(
3          orderInfo: OrderCreateDto): UUID? {
4          // 1. load the customer and the cart
5          // 2. create the order from the cart,
6          //    each line using
7          //    pricing.priceFor(product, customer)
8          // 3. save the order
9          // 4. publish OrderPlaced
10         // 5. return the order id
11     }
12 }
```

The steps of one use case. No business rule of its own.

Domain service

```
1  class PricingService(...) {
2      fun priceFor(product: Product,
3                   customer: Customer): Money {
4          // the loyalty discount
5          // the conversion to the customer's
6          //   currency
7      }
8  }
```

A business rule. It does not load or save anything.

use

Exercise: The discount only once



New rule: customers with ten orders or more still get 5% off, but only once per product.

How should we model the new rule?

- A. `PricingService` keeps a map from each customer to the products they received the discount on
- B. `Customer` holds the ids of the products it received the discount on: `PricingService` checks them, and `Customer` adds the ids when an order is placed
- C. `Product` holds the ids of the customers who received the discount on it, and `PricingService` checks them
- D. `Customer` holds a list of the `Product` objects it received the discount on

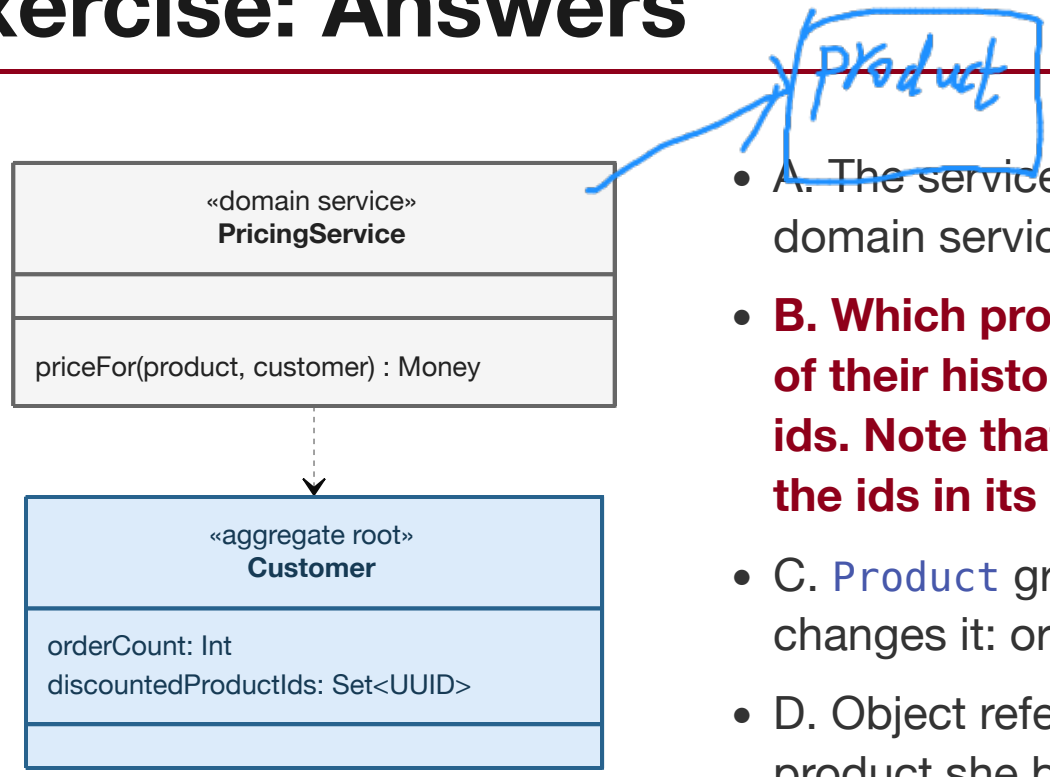
Exercise: The discount only once

Go to wooclap.com and use the code **OCPCYJR**

New rule: customers with ten orders or more still get 5% off, but only once per product. How should we model it?

1. PricingService keeps a map from each customer to the products they received the discount on — 9% — 2
2. **Customer holds the ids of the products it received the discount on: PricingService checks them, and Customer adds the ids when an order is placed** — Correct answer — 83% — 19
3. Product holds the ids of the customers who received the discount on it, and PricingService checks them — 9% — 2
4. Customer holds a list of the Product objects it received the discount on — 0% — 0

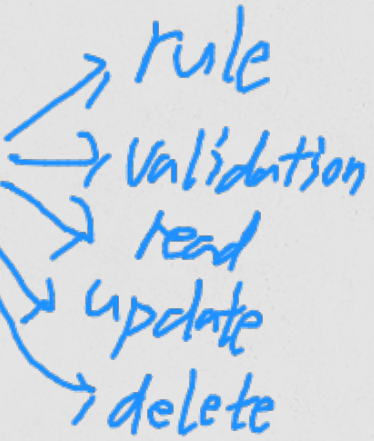
Exercise: Answers



- ~~A. The service now remembers customers between calls: it has state. A domain service stays stateless.~~
- **B. Which products a customer already got the discount on is part of their history, so it belongs to the `Customer` aggregate, as product ids. Note that `PricingService` stays stateless. `Customer` can add the ids in its own transaction, in reaction to `OrderPlaced`.**
- C. `Product` grows with every customer, and every discounted order changes it: orders for a popular product wait on each other.
- D. Object references across aggregates: loading Ana loads every product she bought with the discount.

Structured design

Where to put business logics



Factory

Creating an order

Step 2 of `placeOrder` creates a new order from the cart. Creating an order means:

1. check that the cart has at least one line
2. Create an new order from cart; for each cart line, create an order line at `pricing.priceFor(product, customer)`
3. set the customer and the delivery address
4. assign a new id, a number, the date, and the status `NEW`



Customers also want a “buy again” button that creates a new order from a past one.

The constructor with seven parameters

```
1  // in placeOrder
2  val lines = cart.lines.map { /* order line at pricing.priceFor(...) */ }
3  val order = Order(UUID.randomUUID(), numbers.next(), LocalDate.now(), NEW,
4                    customer.id, address, lines)
5
6  // in buyAgain: the same code, with a past order instead of a cart
7  val lines = pastOrder.lines.map { /* order line at pricing.priceFor(...) */ }
8  val order = Order(UUID.randomUUID(), numbers.next(), LocalDate.now(), NEW,
9                    customer.id, address, lines)
```

Two use cases with almost identical creation logic. If the rules for a new order change, I have to find both.



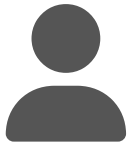
Rules for creating orders change



From now on, a product we no longer sell cannot be ordered.

Done: `placeOrder` now rejects a cart with a discontinued product.





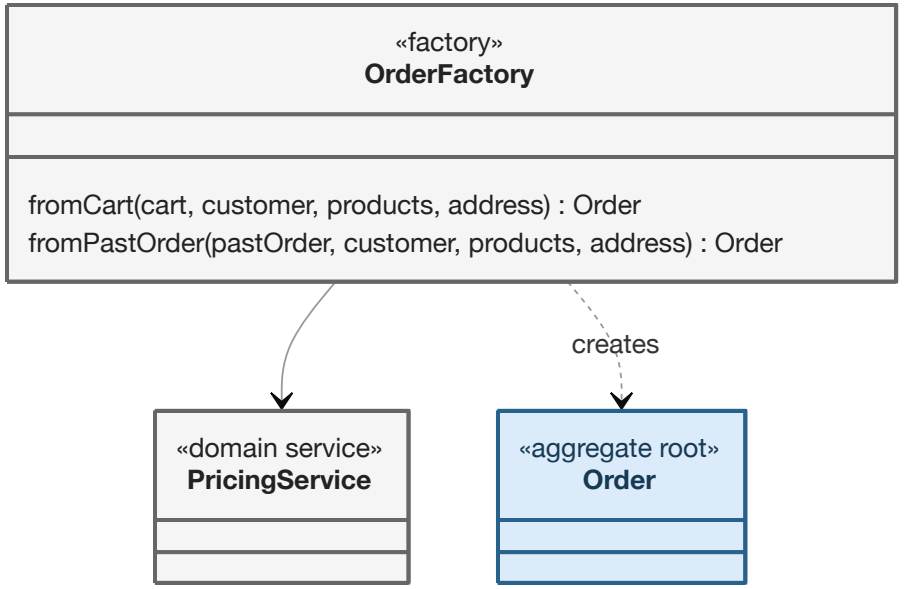
I clicked “buy again” and ordered a mouse you stopped selling last year.

Creating an order has its own rules. They should live in one place, next to the order.

Factory

Factory

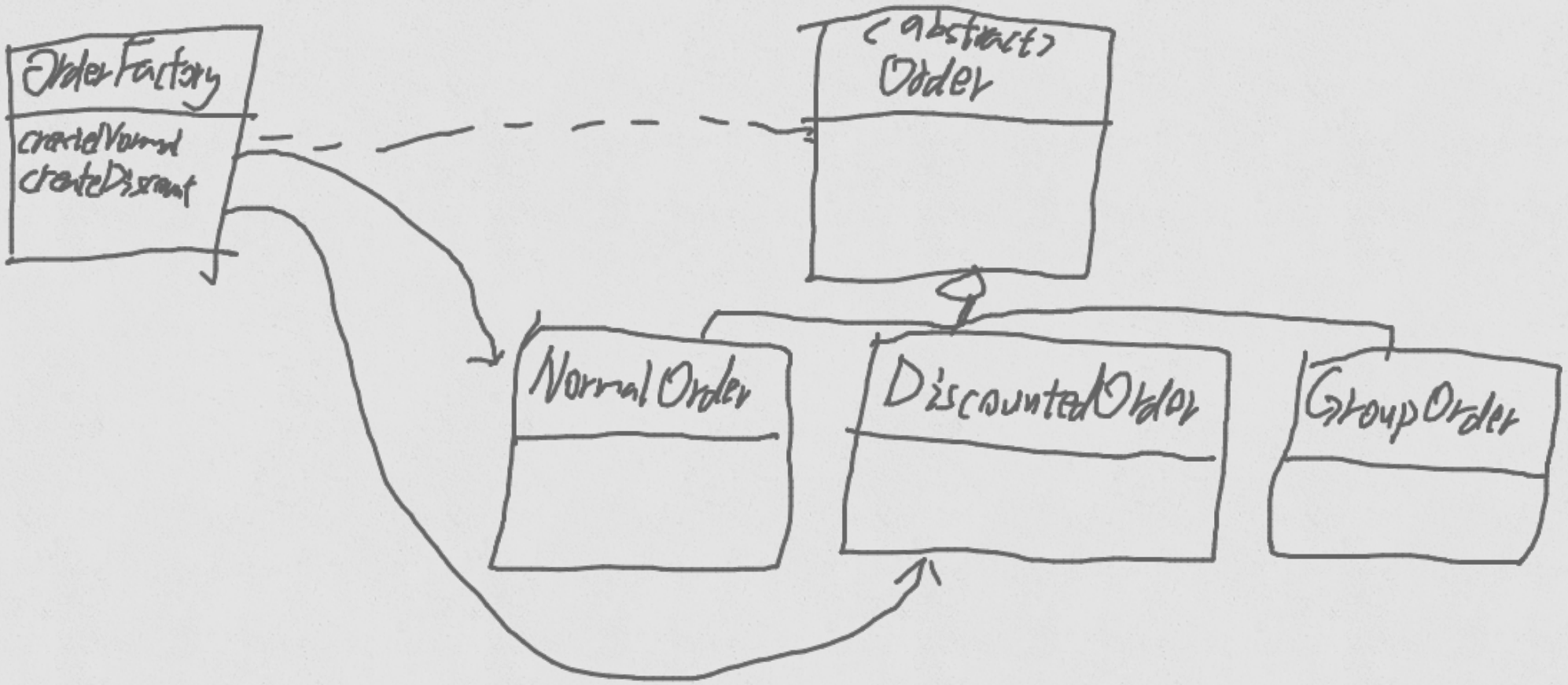
A **factory** creates a complex object, or a whole aggregate, in one step. It hides how the object is assembled, and callers only ever receive a complete, valid object.




```
class OrderFactory(
    private val pricing: PricingService,
    private val numbers: OrderNumbers
) {
    fun fromCart(cart: Cart, customer: Customer,
                 products: Map<UUID, Product>, address: Address) =
        create(cart.lines, customer, products, address)

    fun fromPastOrder(pastOrder: Order, customer: Customer,
                      products: Map<UUID, Product>, address: Address) =
        create(pastOrder.lines, customer, products, address)

    // the common rules for a new order
    private fun create(lines: List<ProductQuantity>, customer: Customer,
                       products: Map<UUID, Product>, address: Address): Order {
        // at least one line, no discontinued product
        // each line at pricing.priceFor(product, customer)
        // a new id, a number, the date, and the status NEW
    }
}
```


OrderFactory
createNormal
createDiscount
<abstract>
Order
NormalOrder
DiscountedOrder
GroupOrder

Repository

Where do orders live between requests?



Step 3 of `placeOrder`: save the order. A bit of SQL right here will do.

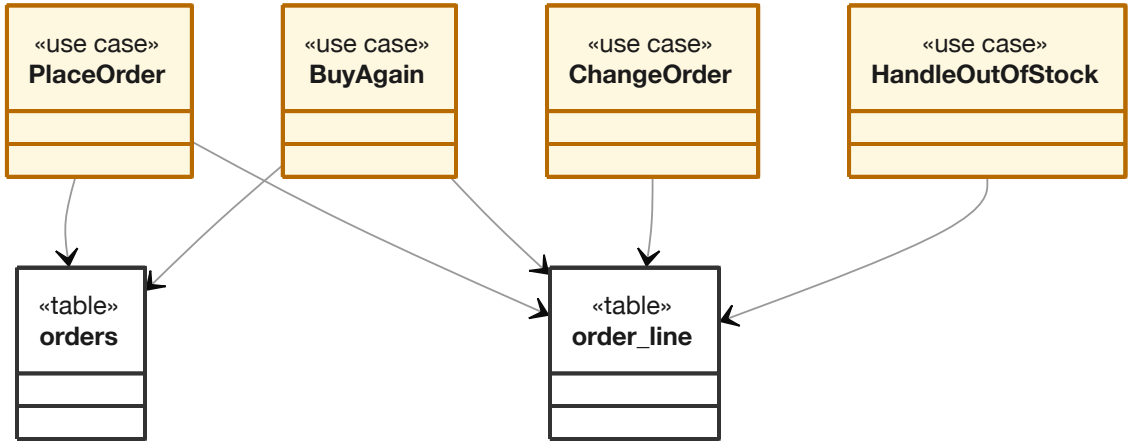
```
1 // in placeOrder
2 db.update("INSERT INTO orders VALUES (...)", order.id, order.customerId, ...)
3 order.lines.forEach {
4     db.update("INSERT INTO order_line VALUES (...)", order.id, it.productId, ...)
5 }
```

Works. The scenario passes.



SQL everywhere

Every use case that touches an order writes its own SQL…




```
1  // in ChangeOrder
2  db.update("UPDATE order_line ...", 3, lineId)
```

Faster than loading the whole order, right?



This changes one order line on its own! The aggregate root never sees it, so the total maybe wrong again. And the use cases now depend on the database tables.

We are moving



Good news: next month, orders move to Firestore.

Then every SQL string, in every use case that touches an order…



And to run the Gherkin scenarios for `placeOrder`, I need a database running. Do we have a test database…?

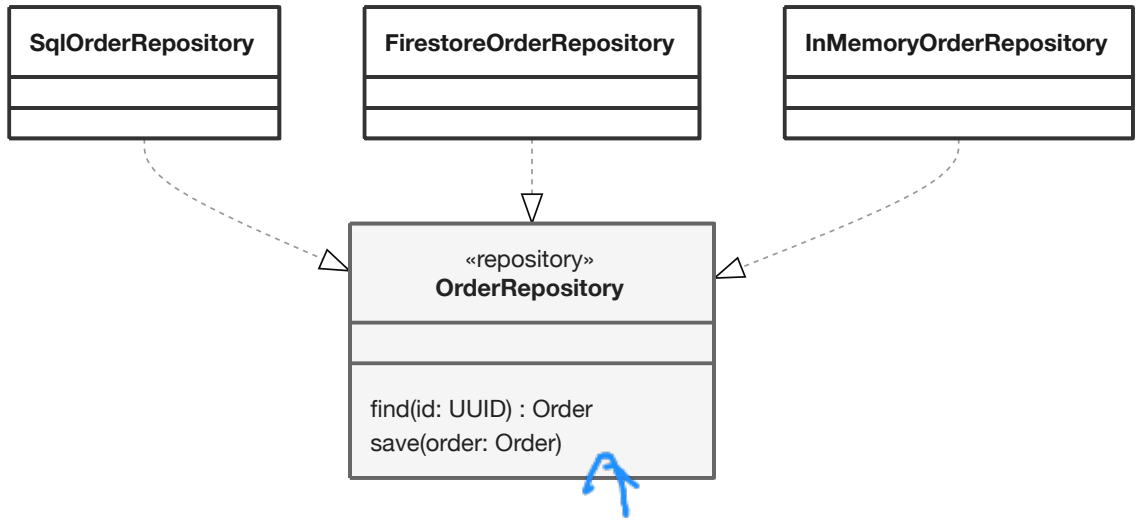


The use cases should not know where, or how, orders are stored.

Repository

Repository

A **repository** lets the domain treat stored aggregates like a collection in memory: add one, find one by its id. It hides how and where they are stored.




```
1  // domain
2  interface OrderRepository {
3      fun find(id: UUID): Order?
4      fun save(order: Order)
5  }
6
7  // infrastructure
8  class FirestoreOrderRepository(...)
9      : OrderRepository { ... }
```

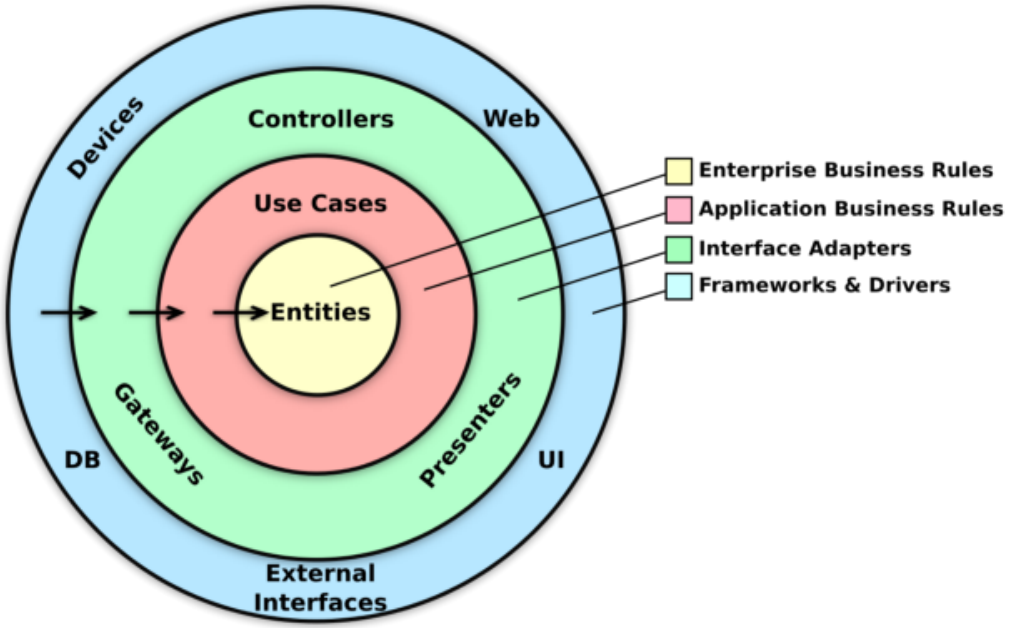
Firestore is one more implementation, and the Gherkin scenarios run on the in-memory one. The use cases do not change.

One repository per aggregate

- `OrderRepository`: yes
- `OrderLineRepository`: no

The repository loads and saves the whole aggregate, through its root.

The interface is in the domain. The implementation is in the infrastructure, so the dependency points inward.

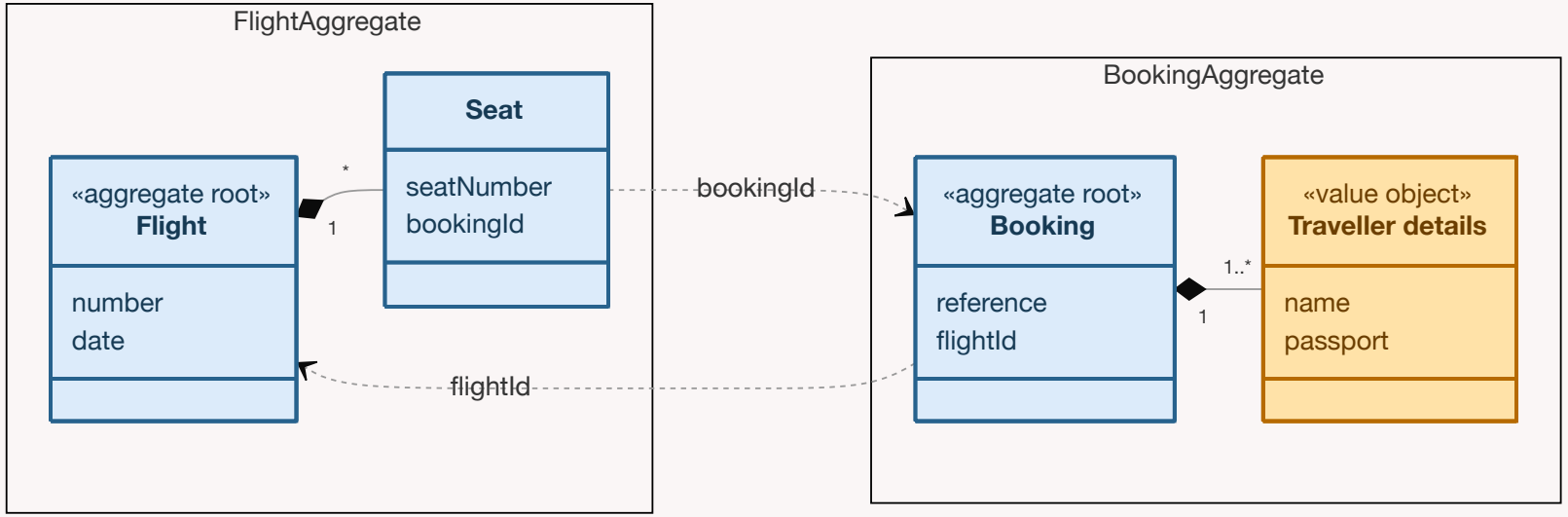


Exercise: Which repositories?

Let's go back to the flight booking system.

For which of these concepts should we create a repository? Select all that apply.

- A. Flight
- B. Seat
- C. Booking
- D. Traveller details



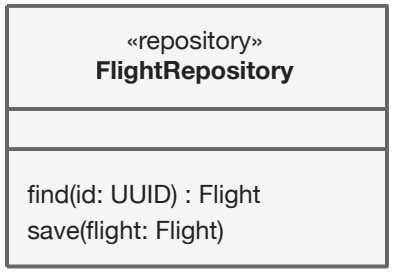
Exercise: Which repositories?

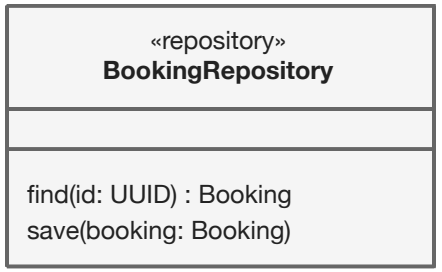
Go to wooclap.com and use the code **OCPCYJR**

Let's go back to the flight booking system. For which of these concepts should we create a repository? Select all that apply.

1	Flight	Correct answer	71% 12
2	Seat		0% 0
3	Booking	Correct answer	94% 16
4	Traveller details		6% 1

Exercise: Answers





One repository per aggregate root.

- **A. Flight: yes. It is the root of its aggregate; the repository loads and saves a flight with all its seats.**
- B. Seat: no. A seat is inside the Flight aggregate. Saving one seat on its own skips the flight's check that two travellers never hold the same seat.
- **C. Booking: yes. It is the root of its aggregate.**
- D. Traveller details: no. A value object inside Booking, saved with its booking.

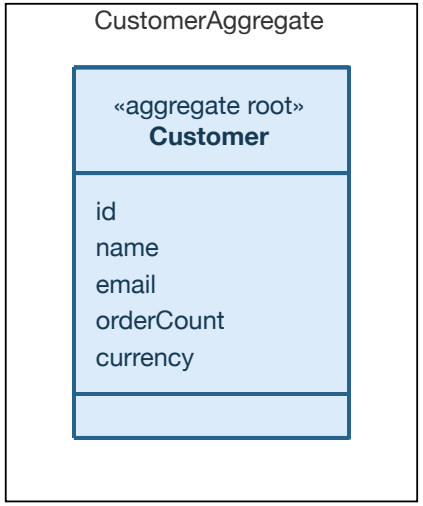
Putting it together

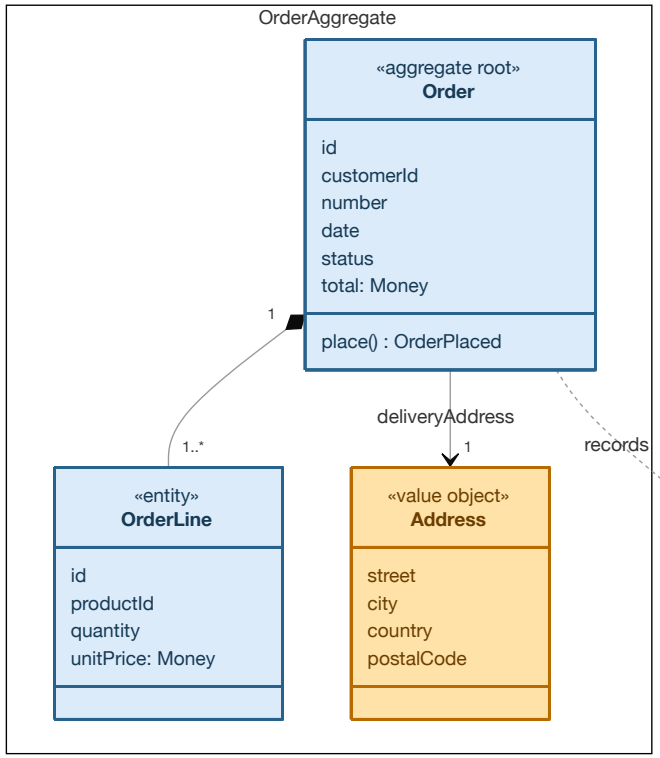
Place an order

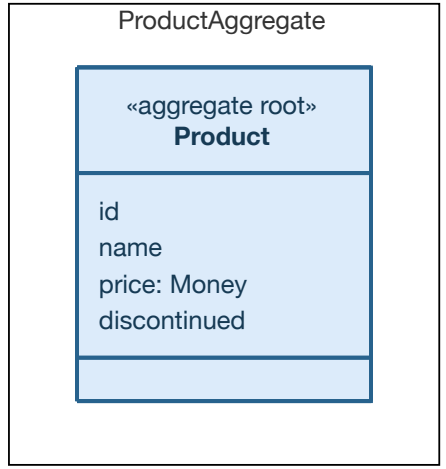
```
class PlaceOrderImpl(...) : PlaceOrder {
    override fun placeOrder(orderInfo: OrderCreateDto): UUID? {
        // 1. load the customer and the cart
        //    repositories: CustomerRepository, CartRepository
        // 2. create the order from the cart, and place it
        //    factory: OrderFactory, with the domain service PricingService
        //    aggregate: Order records the domain event OrderPlaced
        // 3. save the order
        //    repository: OrderRepository
        // 4. publish OrderPlaced
        //    domain event: Stock, Cart, and the email react
        // 5. return the order id
    }
}
```

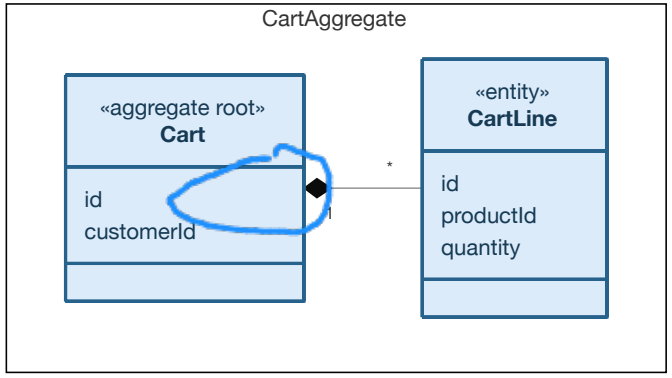
The application command only loads, calls, and saves. Every rule is in the domain.

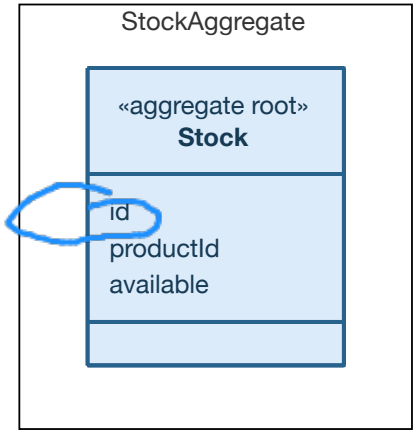
The online shop model





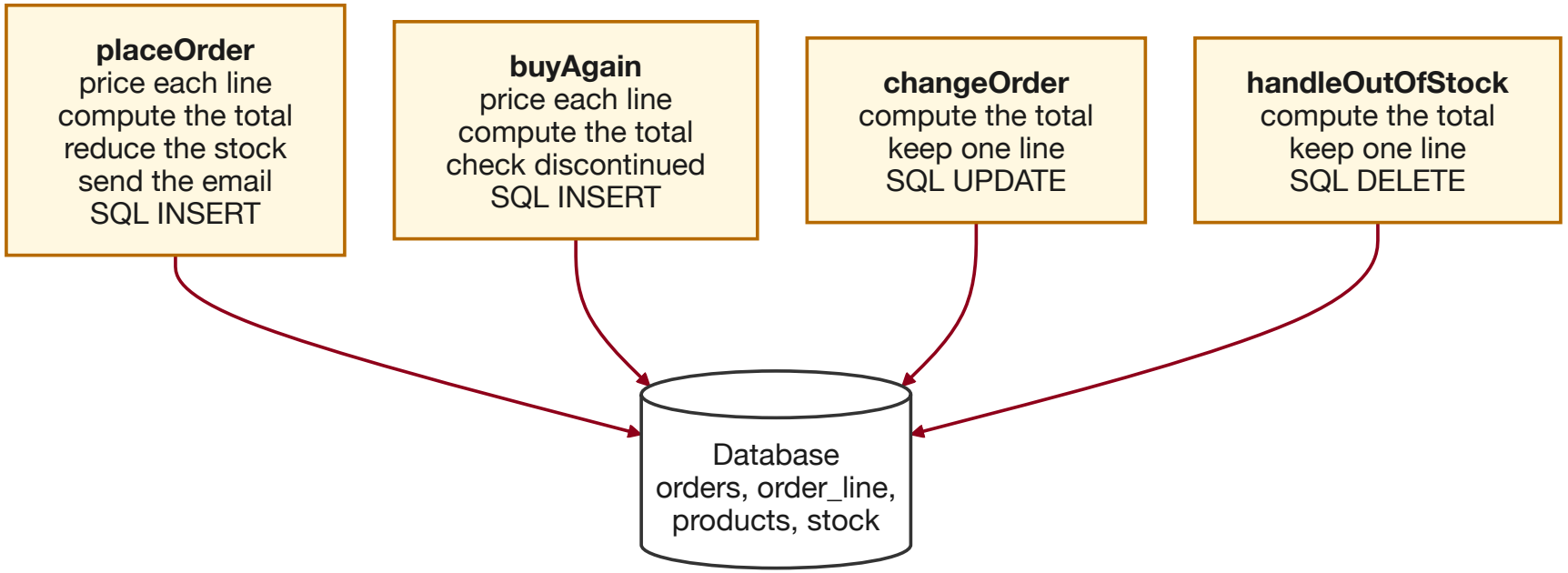






Between aggregates, only ids: customerId, productId

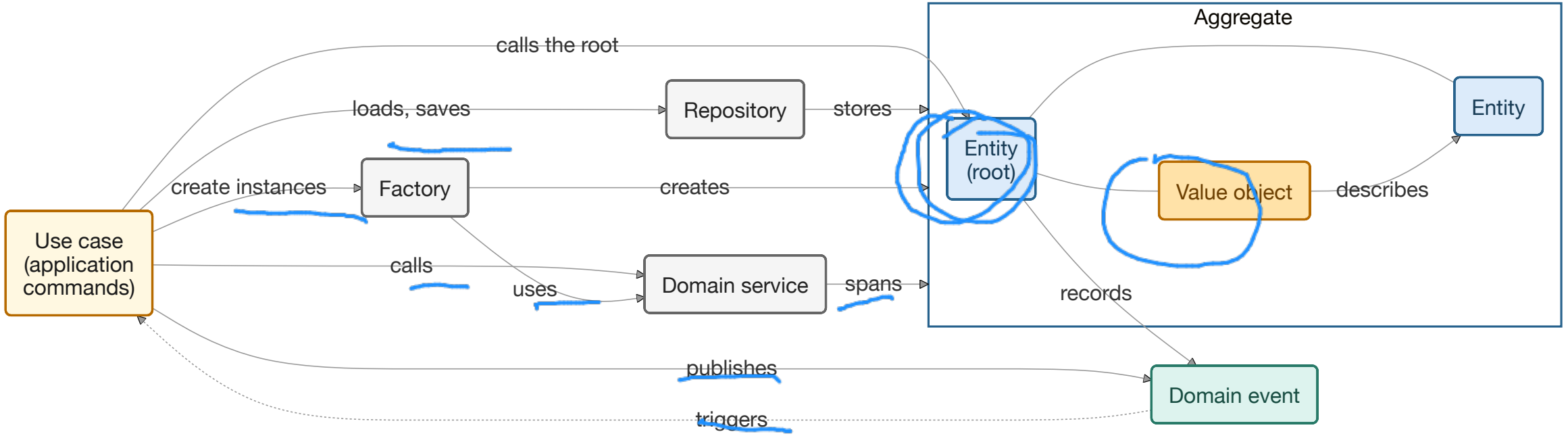
Without structure design



Every rule is copied into the use cases directly interacting with the data. New rule are added in many places, and any change must be repeated in all of them.

Structure design takes the logic out of the use cases and gives each rule a home in the domain: next to it data, under a name domain experts use

With Structure design



Use cases reach the model only through repositories, factories, domain services, aggregate roots, and events

Structure design answers one question: where does each piece of logic belong?

Which pattern when?

When	Pattern
Two objects with the same attributes must still be told apart	Entity
A concept is defined only by its attributes, and its has behavior	Value object
A rule spans several objects, and changing one of them alone can break it	Aggregate
Others react to a change, and the changed object should not directly call each one	Domain event
A rule involves several objects and belongs to none of them	Domain service
Creating an object has its own rules, and more than one caller needs them	Factory
Code needs stored aggregates but should not know how or where they are stored	Repository

Exercise: Where does this logic go?

Match each of the flight booking business logic to a pattern that might hold it.

1. Refuse to give a traveller a seat that is already taken on the flight
2. Compute the price of a booking from the flight's date and the number of free seats left
3. Send email notifications to customer and staff when a flight is cancelled
4. Create a round-trip booking.

- Value object
- Aggregate root
- Domain service
- Domain event
- Factory
- Repository
- Application command

Exercise: Where does this logic go?

Go to wooclap.com and use the code **OCPCYJR**

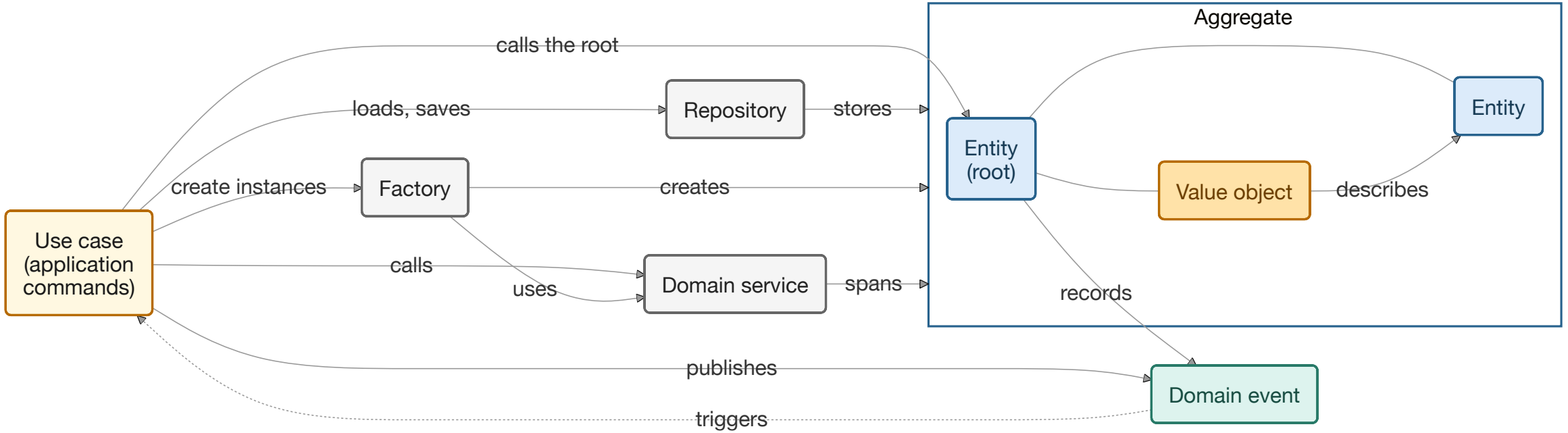
Match each of the flight booking business logic to a pattern that might hold it.

Refuse to give a traveller a seat that is already taken on the flight	1 — ✓ 0% — B	Aggregate root
Compute the price of a booking from the flight's date and the number of free seats left	2 — ✓ 0% — A	Domain service
Send email notifications to customer and staff when a flight is cancelled	3 — ✓ 0% — D	Domain event
Create a round-trip booking.	4 — ✓ 0% — C	Factory

Exercise: Answers

Request	Pattern	Why
No seat taken twice	Aggregate root: `Flight`	Only the flight sees all its seats
Price from date and free seats	Domain service: `FareService`	Needs a booking and a flight; belongs to neither
Emails when a flight is cancelled	Domain event: `FlightCancelled`	Flight records what happened; the email service reacts, and Flight does not know it
Round-trip booking	Factory: `BookingFactory`	Two flights and the same travellers, assembled and checked in one place

Summary


Aggregate
calls the root
loads, saves
Repository
stores
Entity
Entity (root)
create instances
Factory
creates
Value object
describes
Use case (application commands)
calls
uses
Domain service
spans
records
publishes
Domain event
triggers